Universidad / Facultad (completar)

Carrera de Ingeniería (completar)

Laboratorio 8.1: Navegación Multitarea y Arquitectura de Flujo (Login a Dashboard)

Navigation Compose y manejo del Back Stack en la aplicación EcoTrip 2026

Desarrollo de Aplicaciones Móviles · RDA-2 · Grupo 4

James Argüello

Wismar Benítez

Carlos Galarza

Mateo Morales

Ángel Loor

Asignatura (completar)

Docente (completar)

Octubre de 2026

Introducción

Una aplicación real rara vez es una sola pantalla: el usuario se autentica, llega a un panel principal, abre detalles y vuelve. Cada una de esas transiciones modifica la pila de retroceso (*back stack*), la estructura que decide a dónde lleva el botón «Atrás». Si la pantalla de inicio de sesión permanece en esa pila, una persona que ya ingresó puede regresar a ella con un solo gesto, lo que en el mejor caso es confuso y en el peor expone un flujo de autenticación que debería estar cerrado (Android Developers, s. f.-a).

Este laboratorio implementa con Navigation Compose un flujo unidireccional Login → Dashboard sobre la aplicación EcoTrip 2026, controla la pila con *popUpTo* y verifica con el emulador (Android 16, API 36) que, desde el Dashboard, «Atrás» cierra la aplicación en lugar de volver al Login. Se resuelve también el desafío extra de cierre de sesión.

Objetivos

1. Implementar Navigation Compose para estructurar la arquitectura de flujo de la aplicación.

2. Construir un *NavHost* que gestione varios destinos (autenticación y principal).

3. Manipular el *back stack* con *popUpTo* para evitar retornos no deseados.

4. Verificar el comportamiento con evidencia del emulador y con pruebas instrumentadas.

Marco teórico

Componentes de Navigation Compose. El *NavController* mantiene el *back stack* y ejecuta las transiciones; el *NavHost* asocia cada ruta con un *composable* y declara el destino inicial (*startDestination*); la función *composable(ruta)* registra un destino (Android Developers, s. f.-b). La ruta es una cadena en este laboratorio, a diferencia de las rutas tipadas del examen anterior, que usan clases serializables.

Back stack y *popUpTo*. Cada llamada a *navigate* apila un destino y «Atrás» lo desapila. El bloque *navigate(ruta) { popUpTo(otra) { inclusive = true } }* elimina de la pila todos los destinos hasta *otra* y, con *inclusive = true*, también a *otra*, antes de apilar el nuevo destino (Android Developers, s. f.-a). Es la forma declarativa de lo que antes se lograba llamando a *popBackStack()* y luego a *navigate()*. Si la pila queda vacía, «Atrás» ya no tiene un destino interno al que volver y la *Activity* se cierra según las reglas del sistema (Android Developers, s. f.-c).

Estado elevado. Las pantallas reciben lambdas (*onLoginSuccess*, *onLogout*) en lugar del *NavController*. Así no dependen de la navegación, son más fáciles de probar y de previsualizar, y el único lugar que conoce las rutas es el grafo.

Desarrollo

Adaptaciones respecto a la guía

La guía parte de «SafePass 2026»; el repositorio del grupo contiene EcoTrip 2026, de modo que se usó el paquete real *com.example.myapplication* y el tema *MyApplicationTheme*. La dependencia *navigation-compose* ya estaba declarada (versión 2.8.3 en el catálogo, posterior a la 2.8.0 de la guía), por lo que el Paso 1 se verificó y no requirió cambios. Además, se conservó el flujo existente de EcoTrip (formulario y resumen) y se integró como tercer destino, en lugar de eliminarlo.

Paso 2. Pantallas

Se crearon *LoginScreen* y *DashboardScreen* en el paquete *ui.screens* con los textos exigidos por la guía. *LoginScreen* recibe *onLoginSuccess*; *DashboardScreen* recibe *onLogout* (desafío extra) y *onAbrirEcoTrip* para llegar al formulario. Ambas usan los estilos de Material 3 del tema, de modo que heredan Poppins y el color dinámico del laboratorio anterior.

Paso 3. Grafo de navegación (NavGraph.kt)

El archivo *ui/navigation/NavGraph.kt* centraliza la navegación. Los fragmentos clave son los que manipulan la pila:

NavGraph.kt: del Login al Dashboard (Paso 3)

```
        composable("auth") {
            LoginScreen(
                onLoginSuccess = {
                    // POR QUÉ: se navega a 'main' y se elimina 'auth' de la pila, para que
el
                    // botón "Atrás" no devuelva al login.
                    navController.navigate("main") {
                        popUpTo("auth") { inclusive = true }
                    }
                }
            )
        }
```

NavGraph.kt: cierre de sesión (desafío extra)

```
        composable("main") {
            DashboardScreen(
                onLogout = {
                    navController.navigate("auth") {
                        popUpTo("main") { inclusive = true } // Destruye el dashboard
                    }
                },
                onAbrirEcoTrip = { navController.navigate("ecotrip") }
```

```
            )
        }
```

En el primer bloque, *popUpTo("auth") { inclusive = true }* deja la pila con el Dashboard como única entrada. En el segundo, el cierre de sesión hace la operación inversa: apila *auth* y elimina *main*, de modo que tras salir no se puede volver al panel con «Atrás». El listado completo está en el Anexo.

Paso 4. Inyección en MainActivity

En *setContent* se sustituyó la llamada anterior por *AppNavigation*, dentro de *MyApplicationTheme*. El *ViewModel* de EcoTrip se crea una vez en la *Activity* y se pasa al grafo, conservando la resiliencia de estado del examen anterior.

Resultados

Se ejecutó la aplicación en el emulador Pixel 8 con Android 16 (API 36). La Figura 1 reproduce la prueba de fuego del laboratorio.

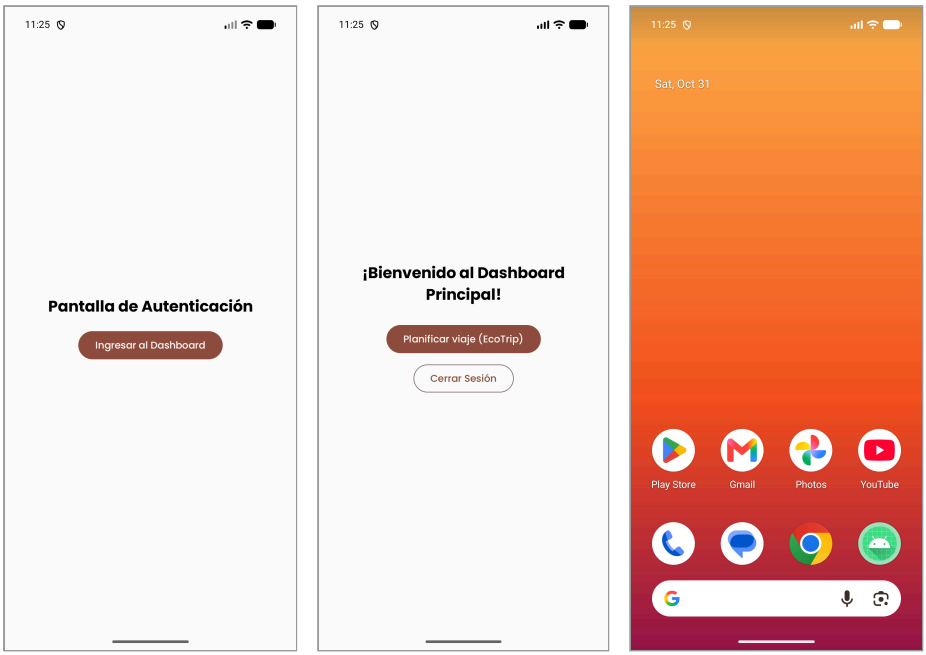


Figura 1. Secuencia de la prueba: (izquierda) pantalla de autenticación al abrir la app; (centro) Dashboard tras pulsar «Ingresar al Dashboard»; (derecha) pantalla de inicio del sistema tras pulsar «Atrás» en el Dashboard.

Los tres estados coinciden con los resultados esperados: al abrir, se ve «Pantalla de Autenticación» y su botón; al pulsarlo, «¡Bienvenido al Dashboard Principal!»; y al presionar «Atrás» el sistema muestra la pantalla de inicio (*Home*) sin pasar de vuelta por el Login (Figura 1, derecha).

Para no depender solo de la imagen, se consultó el estado de las tareas con *adb shell dumpsys activity activities*. Antes de «Atrás», la actividad en primer plano era *com.example.myapplication/.MainActivity*; inmediatamente después, era *NexusLauncherActivity* y *MainActivity* ya no figuraba entre los registros de actividad, es decir, la pila de la aplicación quedó vacía.

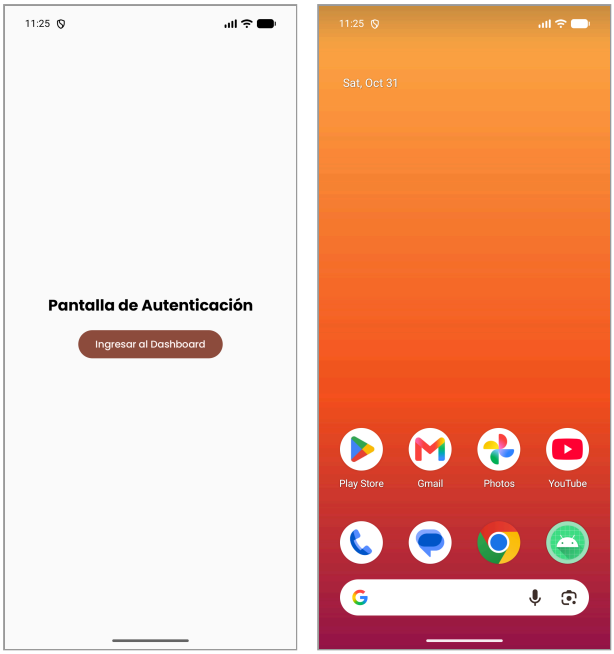


Figura 2. Desafío extra: (izquierda) tras «Cerrar Sesión» se regresa a la pantalla de autenticación; (derecha) «Atrás» desde esa pantalla sale de la aplicación, sin reaparecer el Dashboard.

La Figura 2 comprueba la operación inversa: después de cerrar sesión el Dashboard ya no está en la pila, por lo que «Atrás» no permite reingresar sin pasar de nuevo por el Login.

Pruebas automatizadas

Además de la verificación manual se añadió *NavegacionFlujoTest*, con dos pruebas instrumentadas de Compose (Android Developers, s. f.-d): una verifica que el Login conduzca al Dashboard y que el Login desaparezca; la otra, que «Cerrar Sesión» devuelva al Login y que el Dashboard desaparezca. Como la aplicación ahora inicia en el Login, la prueba de rotación del examen anterior se actualizó para ingresar y abrir el formulario antes de escribir datos. La ejecución de *connectedDebugAndroidTest* en el emulador API 36 terminó con 8 de 8 pruebas aprobadas.

Tabla 1

Resumen de pruebas instrumentadas ejecutadas en el emulador (API 36)

Clase	Pruebas	Resultado
NavegacionFlujoTest (nueva)	2	Aprobadas
ResilienciaTest (rotación actualizada)	5	Aprobadas
ExampleInstrumentedTest	1	Aprobada

Análisis

Por qué *popUpTo* y no solo *navigate*. Sin *popUpTo*, la pila tras el login sería [auth, main]; «Atrás» desapilaría *main* y mostraría *auth* de nuevo. Con *popUpTo("auth") { inclusive = true }* la pila queda [main]. La diferencia se aprecia en un solo gesto, pero cambia la propiedad del flujo: pasa de ser reversible a unidireccional.

Límite de lo que se logró. Este laboratorio simula la autenticación: el botón no valida credenciales y no existe una sesión real. Quitar el Login de la pila evita volver a él, pero no protege el Dashboard por sí solo; un sistema real debe, además, validar el estado de sesión en el servidor o en un repositorio y redirigir cuando expire.

Observación sobre el estado. El *ViewModel* de EcoTrip pertenece a la *Activity*, no al destino «ecotrip». Por eso los datos del formulario sobreviven a un cierre de sesión dentro de la misma ejecución. En esta práctica es un efecto aceptable, pero en una aplicación con datos personales los *ViewModels* de usuario deberían limpiarse al cerrar sesión.

Conclusiones

La gestión del *back stack* es una decisión de diseño, no un detalle técnico: con una sola línea, *popUpTo* convirtió un flujo reversible en uno unidireccional, y la prueba con «Atrás» lo demostró con evidencia visual y con el estado de las tareas del sistema. Navigation Compose concentra esa decisión en un único archivo, lo que hace el flujo legible y fácil de auditar.

Mi valoración es que el patrón de lambdas hacia las pantallas fue lo más rentable del laboratorio: permitió probar el flujo sin tocar la interfaz y ampliarlo con el desafío extra cambiando solo el grafo. También fue instructivo que al integrar el Login como primer destino se rompiera una prueba previa: las pruebas de otros laboratorios dependen del punto de entrada de la aplicación, y cambiarlo exige actualizarlas.

Finalmente, el laboratorio recuerda que ocultar una pantalla no es lo mismo que proteger un recurso. La navegación mejora la usabilidad y evita errores del usuario, pero la seguridad real debe apoyarse en la validación de la sesión.

Recomendaciones

1. Reemplazar las rutas de cadena por rutas tipadas serializables, como ya se hizo en el flujo de EcoTrip, para detectar en compilación los errores de nombre.

2. Decidir el destino inicial según el estado de sesión (por ejemplo, leerlo de DataStore) en lugar de iniciar siempre en el Login.

3. Limpiar los *ViewModels* con datos de usuario al cerrar sesión, o ligarlos al destino mediante un grafo anidado.

4. Validar credenciales y estado de sesión fuera de la interfaz; el *back stack* no es un mecanismo de seguridad.

5. Conservar las pruebas de «Atrás» y de cierre de sesión en la integración continua, para detectar regresiones del flujo.

6. Si el flujo crece, extraer cada sección (autenticación, principal, EcoTrip) a su propio grafo anidado con *navigation()*, de modo que cada una pueda limpiarse como un bloque.

Referencias

Android Developers. (s. f.-a). *Navigate to a destination*. Google. https://developer.android.com/guide/navigation/use-graph/navigate

Android Developers. (s. f.-b). *Navigation with Compose*. Google. https://developer.android.com/develop/ui/compose/navigation

Android Developers. (s. f.-c). *Tasks and the back stack*. Google. https://developer.android.com/guide/components/activities/tasks-and-back-stack

Android Developers. (s. f.-d). *Testing your Compose layout*. Google. https://developer.android.com/develop/ui/compose/testing

Anexo. Código fuente del laboratorio

app/src/main/java/com/example/myapplication/ui/navigation/NavGraph.kt

```
package com.example.myapplication.ui.navigation

import androidx.compose.runtime.Composable
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import com.example.myapplication.navigation.EcoTripNavHost
import com.example.myapplication.ui.screens.DashboardScreen
import com.example.myapplication.ui.screens.LoginScreen
import com.example.myapplication.ui.viewmodel.EcoTripViewModel

@Composable
fun AppNavigation(ecoTripViewModel: EcoTripViewModel) {
    // 1. El NavController es el "conductor" de la aplicación.
    val navController = rememberNavController()

    // 2. El NavHost es el mapa de rutas; startDestination define la primera pantalla.
    NavHost(navController = navController, startDestination = "auth") {

        // Destino 1: Login
        composable("auth") {
            LoginScreen(
                onLoginSuccess = {
                    // POR QUÉ: se navega a 'main' y se elimina 'auth' de la pila, para que el
                    // botón "Atrás" no devuelva al login.
                    navController.navigate("main") {
                        popUpTo("auth") { inclusive = true }
                    }
                }
            )
        }

        // Destino 2: Dashboard
        composable("main") {
            DashboardScreen(
                onLogout = {
                    navController.navigate("auth") {
                        popUpTo("main") { inclusive = true } // Destruye el dashboard
                    }
                },
                onAbrirEcoTrip = { navController.navigate("ecotrip") }
            )
        }

        // Destino 3: flujo EcoTrip (formulario y resumen). "Atrás" regresa al Dashboard.
        composable("ecotrip") {
            EcoTripNavHost(viewModel = ecoTripViewModel)
        }
    }
}
```

app/src/main/java/com/example/myapplication/ui/screens/LoginScreen.kt

```
package com.example.myapplication.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
```

```
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp

@Composable
fun LoginScreen(onLoginSuccess: () -> Unit) {
    // POR QUÉ: la pantalla recibe una lambda y no conoce al NavController; así la UI queda
    // desacoplada de la lógica de navegación.
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Pantalla de Autenticación",
            style = MaterialTheme.typography.titleLarge,
            textAlign = TextAlign.Center
        )
        Spacer(modifier = Modifier.height(16.dp))
        Button(onClick = { onLoginSuccess() }) {
            Text(text = "Ingresar al Dashboard")
        }
    }
}
```

app/src/main/java/com/example/myapplication/ui/screens/DashboardScreen.kt

```
package com.example.myapplication.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp

@Composable
fun DashboardScreen(
    onLogout: () -> Unit,
    onAbrirEcoTrip: () -> Unit
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "¡Bienvenido al Dashboard Principal!",
            style = MaterialTheme.typography.titleLarge,
            textAlign = TextAlign.Center
        )
        Spacer(modifier = Modifier.height(24.dp))
        Button(onClick = { onAbrirEcoTrip() }) {
            Text("Planificar viaje (EcoTrip)")
        }
        Spacer(modifier = Modifier.height(8.dp))
        OutlinedButton(onClick = { onLogout() }) {
            Text("Cerrar Sesión")
        }
```

```
    }
}
```

app/src/main/java/com/example/myapplication/MainActivity.kt

```
package com.example.myapplication

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.filled.Add
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FloatingActionButton
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import com.example.myapplication.ui.navigation.AppNavigation
import com.example.myapplication.ui.theme.MyApplicationTheme
import com.example.myapplication.ui.viewmodel.EcoTripViewModel

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyApplicationTheme {
                val ecoTripViewModel: EcoTripViewModel = viewModel(
                    factory = EcoTripViewModel.factory(application)
                )
                AppNavigation(ecoTripViewModel = ecoTripViewModel)
            }
        }
    }
}

/** Pantalla del Laboratorio 7.2: muestra cómo reacciona la interfaz al Color Dinámico y a Poppins. */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ConfigScreen(onVolver: (() -> Unit)? = null) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = {
                    Text("EcoTrip 2026 - Identidad", style = MaterialTheme.typography.titleLarge)
                },
                navigationIcon = {
                    if (onVolver != null) {
                        IconButton(onClick = onVolver) {
                            Icon(
                                imageVector = Icons.AutoMirrored.Filled.ArrowBack,
```

```
                                contentDescription = "Volver al formulario"
                            )
                        }
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.primaryContainer,
                    titleContentColor = MaterialTheme.colorScheme.onPrimaryContainer,
                    navigationIconContentColor = MaterialTheme.colorScheme.onPrimaryContainer
                )
            )
        },
        floatingActionButton = {
            FloatingActionButton(
                onClick = { /* Acción */ },
                containerColor = MaterialTheme.colorScheme.secondaryContainer,
                contentColor = MaterialTheme.colorScheme.onSecondaryContainer
            ) {
                Icon(Icons.Default.Add, contentDescription = "Agregar")
            }
        }
    ) { innerPadding ->
        Column(
            modifier = Modifier
                .padding(innerPadding)
                .verticalScroll(rememberScrollState())
                .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            Text(
                text = "Bienvenido al futuro de la seguridad móvil.",
                style = MaterialTheme.typography.bodyLarge
            )
            Text(
                text = "Roles de color tomados del fondo de pantalla",
                style = MaterialTheme.typography.titleMedium
            )
            val colores = MaterialTheme.colorScheme
            ColorRole("primary", colores.primary, colores.onPrimary)
            ColorRole("primaryContainer", colores.primaryContainer, colores.onPrimaryContainer)
            ColorRole("secondaryContainer", colores.secondaryContainer, colores.onSecondaryContainer)
            ColorRole("tertiaryContainer", colores.tertiaryContainer, colores.onTertiaryContainer)
            ColorRole("surfaceVariant", colores.surfaceVariant, colores.onSurfaceVariant)
        }
    }
}

@Composable
private fun ColorRole(nombre: String, fondo: Color, contenido: Color) {
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .height(56.dp)
            .background(fondo, RoundedCornerShape(16.dp))
            .padding(horizontal = 16.dp),
        contentAlignment = Alignment.CenterStart
    ) {
        Text(text = nombre, color = contenido, style = MaterialTheme.typography.bodyLarge)
    }
}
```

app/src/androidTest/java/com/example/myapplication/NavegacionFlujoTest.kt

```
package com.example.myapplication

import androidx.compose.ui.test.junit4.createAndroidComposeRule
import androidx.compose.ui.test.onNodeWithText
import androidx.compose.ui.test.performClick
import androidx.test.ext.junit.runners.AndroidJUnit4
import org.junit.Rule
import org.junit.Test
import org.junit.runner.RunWith

```

```
/** Lab 8.1: verifica el flujo Login → Dashboard → Login y que el Back Stack no conserve pantallas. */
@RunWith(AndroidJUnit4::class)
class NavegacionFlujoTest {

    @get:Rule
    val composeRule = createAndroidComposeRule<MainActivity>()

    @Test
    fun login_navegaAlDashboard() {
        composeRule.onNodeWithText("Pantalla de Autenticación").assertExists()
        composeRule.onNodeWithText("Ingresar al Dashboard").performClick()
        composeRule.onNodeWithText("¡Bienvenido al Dashboard Principal!").assertExists()
        composeRule.onNodeWithText("Pantalla de Autenticación").assertDoesNotExist()
    }

    @Test
    fun cerrarSesion_regresaAlLogin() {
        composeRule.onNodeWithText("Ingresar al Dashboard").performClick()
        composeRule.onNodeWithText("Cerrar Sesión").performClick()
        composeRule.onNodeWithText("Pantalla de Autenticación").assertExists()
        composeRule.onNodeWithText("¡Bienvenido al Dashboard Principal!").assertDoesNotExist()
    }
}
```